

이더리움 클라이언트를 엄밀 검증할 수 있을까

송영준, KAIST 전산학부

이 글은 이더리움 코어 개발자 디스코드에 올라온 질문, 즉 하드포크마다 되풀이되는 클라이언트 버그를 엄밀 검증으로 막을 수 있느냐는 물음에 절반만 그렇다고 답한다. 엄밀 검증이 보장하는 것은 코드와 명세의 일치뿐이므로 검증의 사정권은 명세가 그어놓은 선을 넘지 못한다. 작년 9월 Reth의 상태 루트 불일치는 그 선 안쪽에 있어 증명으로 막을 수 있었던 사고다. 반면 작년 12월 Prysm 사고는 판정의 결과가 아니라 판정에 이르는 경로의 비용에서 비롯되었으므로 명세를 아무리 엄밀하게 다시 쓰더라도 잡히지 않는다. 두 사고를 나란히 놓으면 엄밀 검증이 클라이언트 다양성을 대체하지 못한다는 결론이 나온다. AI가 증명 생성을 자동화하더라도 이 경계는 움직이지 않는다. 엄밀 검증은 도입할 가치가 있지만 그 전에 무엇을 보장하지 않는지부터 합의해야 한다.

2026년 9월 2일, 이 글을 작성하기 약 일주일 전, 이더리움 코어 개발자들이 모여 있는 디스코드에 이더리움 재단 보드진의 한 사람인 pcaversaccio가 글을 하나 올렸다. "이더리움 하드포크 시 발생하는 버그를 예방하기 위해 엄밀 검증(formal verification)을 도입할 수 있을까?" [2] 이 질문에 나는 절반만 그렇다고 생각한다. 엄밀 검증은 코드가 명세와 일치하지만 보장한다. 그런데 이더리움 클라이언트를 실제로 무너뜨린 사고는 명세 안과 밖에서 모두 나왔다. 엄밀 검증을 도입하더라도 이더리움이 지금까지 기대어 온 클라이언트 다양성(client diversity)을 줄일 근거는 되지 못한다.

엄밀 검증이 보장하는 것은 코드가 명세와 일치한다는 명제 하나뿐이다. 클라이언트 버그를 예방하는 방법으로 엄밀 검증이 주목받는 이유도 여기에 있다. 명세가 규정한 대로 동작함을 증명으로 못 박을 수 있다면, 하드포크마다 되풀이되는 구현 실수를 배포 전에 걸러낼 수 있다. 이더리움 재단(Ethereum Foundation, 이하 EF) 소속 연구원 Tau는 엄밀 검증에 앞서 형식 명세(formal specification)부터 필요하다고 지적했다. 현재 이더리움 명세는 주로 Python으로 작성되어 있어, 수많은 구현체가 의존하는 명세부터 엄밀 검증에 적합한 형태로 다시 쓰여야 한다.

작년 9월 Reth에서 벌어진 사고는 그 선 안쪽에 있다. 클라이언트 버그는 pcaversaccio가 지적한 것처럼 주로 하드포크 시점에 발생한다. 하드포크마다 각 클라이언트가 프로토콜 변경 사항을 새로 구현해야 하고 이 과정에서 모든 사용자는 그 구현을 담은 소프트웨어 업데이트를 강요받기 때문이다. Rust로 작성한 실행 클라이언트 Reth에서는 메인넷에서 상태 루트 불일치 문제가 발생하여, Reth 사용자들의 노드가 정상적인 블록 검증을 하지 못하고 네트워크에서 분리되는 버그가 있었다 [3]. 상태 루트는 명세가 규정하는 상태 전이 함수의 결과값이다. 그 값이 다르다는 것은 구현이 명세와 어긋났다는 뜻이다. 엄밀 검증의 표적이 바로 이런 지점이다. 명세를 형식화하고 Reth의 상태 전이 구현이 그것과 일치함을 증명했다면 이 사고는 배포 전에 걸렸을 것이다.

반면 작년 12월 Prysm 사고는 명세를 아무리 엄밀하게 다시 쓰더라도 잡히지 않는다. 당시 Prysm 노드들은 동기화가 어긋난 노드가 보낸 투표를 받았다. 이 투표는 이전 상태에 대한 블록을 가리키고 있었다. Prysm은 그것을 검증하려고 그 시점의 상태를 통째로 다시 만들었다. 과거 블록을 다시 처리하고 상태 전이를 다시 계산하는 일이 반복되면서 노드는 제때 일하지 못했다. 검증자들이 놓친 투표 보상은 약 382 ETH에 달했다 [1]. 다만 Prysm이 내린 판정 자체는 틀리지 않았다. 문제의 투표는 명세상 처리되어야 하는 것이 맞고 Prysm도 결국 맞는 결론에 도달했다. 이후 적용된 수정도 판정을 바꾸지 않았다. 이전 상태를 다시 만드는 대신 노드의 현재 상태로 같은 투표를 검증하도록 경로만 바꿨을 뿐이다. 이 사고는 명세 안에서 걸려질 성질의 것이 아니었다. 명세는 어떤 투표가 유효한지를 규정하지, 그 판정에 이르는 경로가 얼마나 비싼지는 규정하지 않는다. 수정 전 코드와 수정 후 코드는 명세를 똑같이 만족한다. 엄밀 검증은 둘을 구별하지 못한다.

엄밀 검증은 클라이언트 다양성을 대체하지 못한다. 지금까지 이더리움은 네트워크 안정성을 위해 클라이언트 다양성을 가장 중요한 미덕으로 삼아왔다. 이더리움은 크게 실행 계층과 합의 계층으로 나뉜다. 계층마다 서로 다른 프로그래밍 언어로 독자적으로 개발하는 클라이언트가 대략 5개씩 존재한다. 하나의 클라이언트에 문제가 발생하더라도 해당 클라이언트가 네트워크의

절대다수를 차지하지 않는 한 네트워크는 정상적으로 동작하고 완결성을 유지한다. Prysm 사고에서 네트워크가 멈추지 않고 참여율 75%를 지켜낸 것도 나머지 클라이언트들이 제 몫을 했기 때문이다. 엄밀 검증은 이 역할을 넘겨받지 못한다. 명세 밖에서 생기는 사고 앞에서 증명은 무력하다. 그때 남은 방어선은 서로 다르게 구현된 클라이언트뿐이다. Lighthouse의 Michael Sproul이 지난 6개월간 엄밀 검증을 적용해 본 뒤 클라이언트 전체가 아니라 일부 부품부터 검증하자고 결론지은 것도 같은 방향을 가리킨다. 검증된 부품과 검증되지 않은 나머지가 한동안 공존한다면, 그 나머지를 감당하는 것은 여전히 다양성이다.

AI가 증명을 자동으로 생성하게 되더라도 이 경계는 움직이지 않는다. 최근 엄밀 검증 논의에 힘이 실린 데에는 AI의 몫이 크다. 기존에 인간이 직접 작성해야 하던 증명을 AI가 자동으로 생성할 가능성이 부각되면서 검증의 가장 큰 비용이던 증명 작성이 곧 싸지리라는 기대가 커졌다. 그러나 AI가 낮추는 것은 증명을 쓰는 비용이지 명세가 다루는 범위가 아니다. Prysm 사고처럼 명세가 애초에 규정하지 않은 영역은 증명 생성이 아무리 저렴해져도 그대로 남는다. 오히려 비용이 싸질수록 위험은 커진다. 검증을 마쳤다는 말이 안전의 보증처럼 읽히면서 다양성을 줄여도 된다는 결론으로 이어질 수 있기 때문이다. Tau가 던진 질문이 여기서 다시 떠오른다. AI가 현재 명세를 기반으로 형식 명세를 작성했다고 하더라도 커뮤니티가 그것을 실제로 신뢰할 수 있는가? 명세를 쓰는 속도가 빨라진다고 해서 그것을 검토하고 권위를 부여하는 속도까지 빨라지지는 않는다.

엄밀 검증은 도입할 가치가 있다. 다만 무엇을 보장하지 않는지를 함께 적어야 한다. 혹자는 엄밀 검증이 만능열쇠이며 10년 뒤에는 모든 사용자가 A부터 Z까지 완전히 검증된 클라이언트를 사용할 수 있으리라 기대한다. 나는 그 방향 자체에는 동의한다. 명세 적합성 버그는 실제로 존재하고 Reth 사고가 그 증거다. 그런 버그를 증명으로 막을 수 있다면 막는 편이 낫다. 검증된 클라이언트가 나온 뒤에도 다양성은 남아 있어야 한다. 엄밀 검증과 클라이언트 다양성은 같은 일을 두 번 하는 것이 아니라 서로 다른 층위를 맡는다. 엄밀 검증은 명세 안쪽을 맡고 클라이언트 다양성은 그 바깥을 맡는다. 현재 엄밀 검증에 대한 이더리움 커뮤니티의 관심은 어느 때보다 뜨겁다. 그에 비해 엄밀 검증이 무엇을 보장하지 않는지를 따지는 논의는 아직 부족하다. 도입을 논하기 전에 그 경계부터 합의해야 한다.

References

- [1] Offchain Labs. 2025. Fusaka Mainnet Prysm Incident. Prysm documentation, Mainnet Postmortems. <https://prysm.offchainlabs.com/docs/misc/mainnet-postmortems/#fusaka-mainnet-prysm-incident> Accessed: 2026-09-11.
- [2] pcaversaccio. 2026. Can we adopt formal verification to prevent bugs at Ethereum hard forks? Message on the Ethereum R&D Discord server. <https://discord.com/channels/595666850260713488/1544664467076292709> Accessed: 2026-09-11.
- [3] Reth Team. 2025. Incident Post-Mortem: Reth Mainnet State Root Mismatch. Incident post-mortem. <https://laced-king-de5.notion.site/Incident-Post-Mortem-Reth-Mainnet-State-Root-Mismatch-26732f2c348480dea8b8c2a8753696dc> Accessed: 2026-09-11.